



Report date: 2026-08-18T06:38:20.241206608Z

User ID: 632291707661

AWS Account ID: 632291707661

Table of Contents

- 1 High level overview..... 2
 - 1.1 Overview..... 2
 - 1.2 Key features..... 2
- 2 Input output..... 2
- 3 Data flow and dependencies..... 3
 - 3.1 Data flow..... 3
 - 3.2 Data dependencies..... 4
- 4 Error handling..... 5

Documentation

1. High level overview

1.1. Overview

This program operates within a utility or subscription-based billing domain, serving as a core batch processing component in the monthly invoicing workflow. It reads customer and usage data to produce itemized invoices, functioning as the primary billing engine that transforms raw consumption records into financial charges. The program solves the problem of automating monthly bill generation by applying tiered pricing, volume and customer-type discounts, tax calculations, and carrying forward previous balances to produce a final amount owed for each active customer.

1.2. Key features

Feature	Description
Tiered Usage Pricing	Applies three rate tiers based on consumption volume: \$0.50/unit for up to 500 units, \$0.45/unit up to 1,000 units, and \$0.40/unit beyond 1,000 units
Volume Discount	Automatically applies a 10% discount on usage charges when a customer's consumption exceeds 1,000 units in a billing month
Premium Customer Discount	Applies an additional 5% discount on usage charges for customers classified as PREMIUM type, stackable with the volume discount
Minimum Charge Enforcement	Ensures a minimum billable amount of \$25.00 is applied even when discounts reduce the charge below that threshold
Tax Calculation	Computes tax on the taxable amount using a customer-specific tax rate stored in the customer record
Previous Balance Rollover	Incorporates each customer's outstanding previous balance into the final invoice total to reflect cumulative amounts owed
Active Customer Validation	Skips invoice generation for inactive customers and logs an error message, ensuring only active accounts are billed
Invoice File Generation	Produces a pipe-delimited invoice output file containing invoice number, customer details, usage, charges, discounts, tax, and total for downstream processing
Sequential Customer Lookup	Reopens and re-scans the customer file for each usage record to match customer details, supporting a simple flat-file architecture without indexed access

2. Input output

Name	Type	Description
CUSTOMER-FILE	Read	Customer master file (CUSTOMER.dat) - read sequentially to look up customer details including ID, name, type, status, tax rate, and previous balance for each usage record being processed
USAGE-FILE	Read	Usage data file (USAGE.dat) - read sequentially to retrieve customer usage records containing customer ID, units consumed, and billing month, which drive the billing process
INVOICE-FILE	Write	Invoice output file (INVOICE.dat) - written with generated invoice records containing invoice number, customer ID, customer name, usage units, usage charge, discount, tax, previous balance, and final total amount due

3. Data flow and dependencies

3.1. Data flow

Step	Process	Description	Records
1	Open Files	Opens CUSTOMER-FILE and USAGE-FILE for input, and INVOICE-FILE for output	[CUSTOMER-RECORD, USAGE-RECORD, INVOICE-RECORD]
2	Read First Usage Record	Reads the first record from USAGE-FILE to prime the processing loop, populating customer ID, usage units, and billing month	[USAGE-CUST-ID, USAGE-UNITS, USAGE-MONTH]
3	Begin Usage Processing Loop	Iterates over each usage record until end-of-file is reached on USAGE-FILE	[WS-CURRENT-CUSTOMER-ID, WS-USAGE-UNITS, WS-BILLING-MONTH]
4	Find Customer Record	Closes and reopens CUSTOMER-FILE from the beginning for each usage record, performing a sequential scan to find the matching customer by CUST-ID	[CUST-ID, WS-CURRENT-CUSTOMER-ID, WS-FOUND-CUSTOMER, WS-END-CUSTOMER]
5	Validate Customer Status	Checks that the matched customer record has a status of ACTIVE before proceeding with billing; logs an error to console if inactive or not found	[CUST-STATUS, CUST-ID]
6	Calculate Usage Charge	Applies a tiered rate to usage units: 0.50 per unit for up to 500 units, 0.45 for 501-1000 units, and 0.40 for over 1000 units	[WS-USAGE-UNITS, WS-RATE, WS-USAGE-CHARGE]
7	Apply Volume Discount	If usage units exceed the volume threshold of 1000, applies a 10% discount to the usage charge	[WS-USAGE-UNITS, WS-VOLUME-THRESHOLD, WS-USAGE-CHARGE, WS-VOLUME-DISCOUNT-RATE, WS-DISCOUNT]
8	Apply Premium Customer Discount	If the customer type is PREMIUM, adds an additional 5% discount on top of any existing volume discount	[CUST-TYPE, WS-USAGE-CHARGE, WS-PREMIUM-DISCOUNT-RATE, WS-DISCOUNT]

Step	Process	Description	Records
9	Enforce Minimum Charge	Computes the taxable amount as usage charge minus discounts; if the result is below the minimum charge of \$25.00, the minimum charge is used instead	[WS-USAGE-CHARGE, WS-DISCOUNT, WS-MINIMUM-CHARGE, WS-TAXABLE-AMOUNT]
10	Calculate Tax	Applies the customer-specific tax rate from the customer record to the taxable amount to derive the tax amount	[WS-TAXABLE-AMOUNT, CUST-TAX-RATE, WS-TAX]
11	Calculate Final Invoice Amount	Sums the taxable amount, tax, and the customer's previous balance to produce the final amount due	[WS-TAXABLE-AMOUNT, WS-TAX, CUST-PREV-BALANCE, WS-FINAL-AMOUNT]
12	Generate Invoice Number	Constructs a unique invoice number by concatenating the billing month and customer ID separated by a hyphen	[WS-BILLING-MONTH, CUST-ID, WS-INVOICE-NUMBER]
13	Format Invoice Output Line	Formats all billing fields into display-ready numeric formats and concatenates them into a pipe-delimited output line of up to 180 characters	[WS-INVOICE-NUMBER, CUST-ID, CUST-NAME, WS-USAGE-UNITS, WS-DISPLAY-CHARGE, WS-DISPLAY-DISCOUNT, WS-DISPLAY-TAX, WS-DISPLAY-BALANCE, WS-DISPLAY-TOTAL, WS-OUTPUT-LINE]
14	Write Invoice Record	Writes the formatted output line to INVOICE-FILE and displays the invoice number to the console	[INVOICE-RECORD, WS-OUTPUT-LINE, WS-INVOICE-NUMBER]
15	Read Next Usage Record	Reads the next record from USAGE-FILE to continue the processing loop or sets end-of-file flag to terminate	[USAGE-CUST-ID, USAGE-UNITS, USAGE-MONTH, WS-END-USAGE]
16	Close Files and Terminate	Closes all three files after the usage loop completes, displays a completion message, and stops the program	[CUSTOMER-FILE, USAGE-FILE, INVOICE-FILE]

3.2. Data dependencies

Dependency	Description
Sequential customer file re-scan per usage record	CUSTOMER-FILE is closed and reopened from the beginning for every usage record processed. This means customer lookup is $O(n*m)$ and the customer file must remain accessible and unchanged throughout the entire run. Any external modification to CUSTOMER.dat during execution could produce inconsistent results.
Usage record drives customer lookup	The customer ID read from USAGE-FILE (USAGE-CUST-ID) is the sole key used to locate the corresponding CUSTOMER-FILE record. If a usage record references a customer ID not present in CUSTOMER-FILE, billing is skipped entirely for that usage record and only a console error is produced.
CUST-STATUS must be ACTIVE for billing to proceed	Even if a customer is found in CUSTOMER-FILE, billing calculation and invoice generation are gated on CUST-STATUS equaling the literal 'ACTIVE'. Inactive customers produce a console error but no invoice, creating a silent revenue gap if not monitored.

Dependency	Description
CUST-TYPE controls premium discount eligibility	The additional 5% premium discount in CALCULATE-BILL depends on CUST-TYPE equaling the literal 'PREMIUM'. The field is 10 characters wide, so trailing spaces or case differences would cause the condition to fail silently.
WS-DISCOUNT accumulation order	The volume discount is calculated first and stored in WS-DISCOUNT. The premium discount is then added to the existing WS-DISCOUNT value. If the volume discount block is not entered (units <= threshold), WS-DISCOUNT remains zero before the premium discount is applied, which is the correct behavior, but the sequential dependency means reordering these blocks would change results.
CUST-PREV-BALANCE included in final amount without validation	The previous balance from CUSTOMER-FILE is added directly to WS-FINAL-AMOUNT without any range check or sign validation. A corrupt or negative CUST-PREV-BALANCE value would silently distort the invoice total.
WS-OUTPUT-LINE fixed at 180 characters	The STRING operation concatenates all invoice fields into WS-OUTPUT-LINE which is defined as PIC X(180). If the combined length of all fields exceeds 180 characters, data will be silently truncated in the written invoice record.
INVOICE-FILE opened once for the entire run	INVOICE-FILE is opened once at the start and all invoice records are appended sequentially. There is no error handling for write failures; a disk-full or I/O error would cause an abnormal termination with no recovery mechanism.
Minimum charge overrides discount logic	If the net charge after discounts falls below \$25.00, the taxable amount is set to the minimum charge, effectively ignoring the computed discount. Tax and previous balance are still applied on top of this minimum, meaning the final amount can still exceed the minimum charge.
No duplicate usage record detection	There is no check for duplicate USAGE-CUST-ID and USAGE-MONTH combinations in USAGE-FILE. If the same customer appears twice for the same month, two separate invoices will be generated without any warning.

4. Error handling

Category	Description
Customer Lookup Errors	When a customer ID from the usage file cannot be matched in the customer file, the program displays an error message to the console (DISPLAY 'ERROR: CUSTOMER NOT FOUND - ' WS-CURRENT-CUSTOMER-ID) and skips processing for that usage record, continuing to the next one without halting execution.
Customer Status Errors	When a matched customer record has a status other than 'ACTIVE', the program displays an error message to the console (DISPLAY 'ERROR: CUSTOMER IS NOT ACTIVE - ' WS-CURRENT-CUSTOMER-ID) and skips invoice generation for that record, continuing to the next usage record without halting execution.
File Operation Errors	There is no explicit error handling for file open, close, read, or write operations. No FILE STATUS clauses are defined in the FILE-CONTROL section, and no INVALID KEY, ON EXCEPTION, or status code checks are present. File errors would cause unhandled runtime abends.

Category	Description
End-of-File Handling	End-of-file conditions are handled gracefully for both the usage file and the customer file using AT END clauses on READ statements, setting boolean flags (END-OF-USAGE, END-OF-CUSTOMER) to terminate their respective processing loops cleanly.
Minimum Charge Enforcement	A business-rule guard is applied during billing calculation: if the computed charge after discounts falls below the minimum charge threshold (25.00), the taxable amount is set to the minimum charge value, preventing undercharging rather than raising an error.
Data Validation Errors	There is no input data validation for fields such as USAGE-UNITS, CUST-TAX-RATE, CUST-PREV-BALANCE, or CUST-TYPE beyond the active status check. Invalid or malformed data in these fields would propagate silently into calculations without any error reporting or rejection logic.
Arithmetic Errors	No ON SIZE ERROR clauses are used on any COMPUTE statements. Numeric overflow in fields such as WS-USAGE-CHARGE, WS-FINAL-AMOUNT, or WS-TAX would result in silent truncation or a runtime abend depending on the COBOL runtime environment.
Invoice Write Errors	The WRITE statement for INVOICE-RECORD has no INVALID KEY or exception handling clause. Any failure during invoice file writing would result in an unhandled runtime error with no diagnostic message or recovery logic.